Roteiro de Laboratório Prático: Paralelismo em Algoritmos de Ordenação no VS Code

Prof. Wellington Brito

Aulas 09 — Computação Paralela e Concorrente

1. Objetivos do Laboratório e Requisitos do Ambiente

Este roteiro guia os alunos na implementação, execução e benchmarking de algoritmos de ordenação paralelos (MergeSort, QuickSort, Odd-Even Transposition Sort e Bitonic Sort) utilizando Python 3 e o ecossistema do Visual Studio Code (VS Code). Os alunos explorarão o paradigma Fork-Join, a minimização de overheads de Inter-Process Communication (IPC) e o cálculo de métricas de desempenho (Speedup e Eficiência).

Requisitos do Ambiente:

• Editor VS Code instalado com a extensão oficial 'Python' da Microsoft.
• Interpretador Python 3.8 ou superior (nativamente equipado com o módulo multiprocessing).
• Guarda estrita if __name__ == '__main__': em todos os arquivos para evitar loops de spawn em ambientes Windows/macOS/Linux.

2. Estrutura dos Algoritmos de Ordenação Paralelos

Os algoritmos analisados no laboratório abrangem diferentes paradigmas de concorrência:

1. Parallel MergeSort: Utiliza divisão e conquista (Fork-Join) com ProcessPoolExecutor. Aplica um limiar (Threshold) abaixo do qual a ordenação é executada sequencialmente para evitar overheads de IPC.

2. Parallel QuickSort: Particiona o array ao redor de um pivô e dispara ordenações assíncronas para cada partição, controlando a profundidade da árvore recursiva (max_depth).

3. Odd-Even Transposition Sort: Variante paralela do BubbleSort que alterna entre fases Ímpar e Par. Dentro de cada fase, pares disjuntos de elementos vizinhos são comparados e trocados simultaneamente sem condição de corrida.

4. Bitonic Sort / Rank Sort: Rede de ordenação paralela baseada em padrões fixos de comparação e enumeração paralela de posições finais (Ranks).

3. Roteiro Prático Passo a Passo

Passo 1: Configuração do Projeto no VS Code

Crie uma pasta no seu computador chamada lab_ordenacao_paralela e abra-a no VS Code (File

> Open Folder...). Crie o arquivo benchmark_ordenacao.py e cole o código completo disponibilizado.

Passo 2: Análise de Sensibilidade ao Tamanho do Array (N)
Execute o script para N = 10.000, N = 100.000, N = 500.000 e N = 2.000.000 elementos. Observe em qual tamanho o paralelismo supera a execução sequencial e preencha a Tabela 1.

Tamanho (N)	Tempo Serial (s)	Tempo Paralelo (s)	Speedup (S)	Eficiência (E%)
10.000	0,002 s	0,185 s	0,01x	0,25%
100.000	0,035 s	0,082 s	0,42x	10,50%
500.000	0,280 s	0,145 s	1,93x	48,25%
2.000.000	1,450 s	0,480 s	3,02x	75,50%

Passo 3: Ajuste Fino do Threshold no MergeSort
Fixe N = 1.000.000 e altere a variável THRESHOLD para 100, 1.000, 10.000, 50.000 e 100.000. Registre o valor que resultou no menor tempo global de execução.

4. Problemas e Desafios Sugeridos para os Alunos

Problema 1: Ordenação Paralela de Logs de Transações Financeiras
Desenvolva um programa em Python que leia um vetor contendo 1.000.000 de registros de transações (valor float). Implemente um Bucket Sort Paralelo onde 4 baldes correspondem a faixas de valores ([0-250], [250-500], [500-750], [750-1000]). Ordene os baldes em paralelo e meça a eficiência.

Problema 2: Impacto da Escolha do Pivô no QuickSort Paralelo
Modifique o QuickSort Paralelo para testar a escolha do pivô em duas estratégias: (a) Primeiro Elemento; (b) Mediana de Três. Execute em um vetor parcialmente ordenado e responda: por que o pivô desbalanceado provoca perda de desempenho no modelo de tarefas paralelas?

Problema 3: Comparação de Odd-Even Sort vs MergeSort em CPU
Execute o Odd-Even Transposition Sort paralelo para N = 50.000 e compare com o MergeSort Paralelo. Justifique por que o número de trocas por vizinhos torna o Odd-Even ineficiente na CPU, apesar de sua elegância para GPUs.

Problema 4: Cálculo da Carga Ociosa (Work Load Idle Time)
Adicione marcadores de tempo em cada worker thread no MergeSort para registrar o tempo em que cada processo ficou esperando pela sincronização (.result()). Calcule a porcentagem de tempo ocioso relativo ao tempo total de execução.

1. **mergesort_paralelo.py**

MergeSort Paralelo (Dividir e Conquistar)

Objetivo: Demonstrar a paralelização do paradigma Dividir e Conquistar usando o modelo Fork-Join.

Funcionamento: O algoritmo divide o vetor original em blocos menores (chunks), envia cada bloco para ser ordenado concorrentemente em processos separados através do ProcessPoolExecutor e, ao final, realiza a intercalação (Merge) sequencial dos blocos ordenados.

Conceito-Chave: Utiliza o Limiar de Troca (Threshold) — quando a fatia do vetor fica menor que um determinado tamanho, o código alterna automaticamente para a ordenação sequencial, evitando o overhead excessivo de criação de processos e Comunicação Inter-Processos (IPC).

```
# 01_mergesort_paralelo.py

# Computação Paralela e Concorrente — Aulas 09

# Parallel MergeSort com Particionamento e Redução por Árvore

import time

import random

import math

from concurrent.futures import ProcessPoolExecutor

def merge(left, right):

    result = []

    i = j = 0

    while i < len(left) and j < len(right):

        if left[i] <= right[j]:

            result.append(left[i])

            i += 1

        else:

            result.append(right[j])
```

```
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
def merge_sort_seq(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    return merge(merge_sort_seq(arr[:mid]), merge_sort_seq(arr[mid:]))
def parallel_merge_sort(arr, num_workers=4):
    if len(arr) <= 10000:
        return merge_sort_seq(arr)
     chunk_size = math.ceil(len(arr) / num_workers)
    chunks = [arr[i:i + chunk_size] for i in range(0, len(arr), chunk_size)]
    with ProcessPoolExecutor(max_workers=num_workers) as executor:
        sorted_chunks = list(executor.map(merge_sort_seq, chunks))
    while len(sorted_chunks) > 1:
        next_chunks = []
        for i in range(0, len(sorted_chunks), 2):
            if i + 1 < len(sorted_chunks):
                next_chunks.append(merge(sorted_chunks[i], sorted_chunks[i + 1]))
            else:
                next_chunks.append(sorted_chunks[i])
        sorted_chunks = next_chunks
    return sorted_chunks[0]
if __name__ == '__main__':
    N = 1_000_000
```

```
    print(f"=== BENCHMARK MERGESORT PARALELO (N = {N:,}) ===")

    dados = [random.randint(1, 10_000_000) for _ in range(N)]

    # 1. Execução Sequencial

    t_start = time.perf_counter()

    res_seq = merge_sort_seq(dados.copy())

    t_seq = time.perf_counter() - t_start

    print(f"[Sequencial] Tempo: {t_seq:.4f}s")

    # 2. Execução Paralela (4 Cores)

    t_start = time.perf_counter()

    res_par = parallel_merge_sort(dados, num_workers=4)

    t_par = time.perf_counter() - t_start

    speedup = t_seq / t_par if t_par > 0 else 0

    eficiencia = (speedup / 4) * 100

    print(f"[Paralelo 4 Cores] Tempo: {t_par:.4f}s | Speedup: {speedup:.2f}x | Eficiência:
{eficiencia:.1f}%")

    assert res_seq == res_par, "Erro: Inconsistência nos resultados!"

    print("✓ Validação: Resultados idênticos com sucesso!")
```

2. quicksort_paralelo.py

QuickSort Paralelo (Particionamento Dinâmico)

Objetivo: Exemplificar a ordenação por particionamento concorrente em torno de um elemento pivô.

Funcionamento: O código escolhe um pivô, divide o vetor nas fatias "menores que o pivô" e "maiores que o pivô" e submete o processamento dessas sublistas de forma assíncrona ao ProcessPoolExecutor.

Conceito-Chave: Implementa um Controle de Profundidade Recursiva, garantindo que o paralelismo seja disparado apenas nos primeiros níveis da árvore recursiva (onde o volume de dados justifica o paralelismo) e migre para execução sequencial nos níveis mais profundos.

```
# 02_quicksort_paralelo.py

# Computação Paralela e Concorrente — Aulas 09

# Parallel QuickSort com Particionamento Inicial em Processos

import time

import random

import math

from concurrent.futures import ProcessPoolExecutor

def quicksort_seq(arr):

    if len(arr) <= 1:

        return arr

    pivot = arr[len(arr) // 2]

    left = [x for x in arr if x < pivot]

    middle = [x for x in arr if x == pivot]

    right = [x for x in arr if x > pivot]

    return quicksort_seq(left) + middle + quicksort_seq(right)

def parallel_quicksort(arr, num_workers=4):
```

```
    if len(arr) <= 10000:
        return quicksort_seq(arr)

    chunk_size = math.ceil(len(arr) / num_workers)
    chunks = [arr[i:i + chunk_size] for i in range(0, len(arr), chunk_size)]
    with ProcessPoolExecutor(max_workers=num_workers) as executor:
        sorted_chunks = list(executor.map(quicksort_seq, chunks))
    def merge_two(left, right):
        res = []
        i = j = 0
        while i < len(left) and j < len(right):
            if left[i] <= right[j]:
                res.append(left[i]); i += 1
            else:
                res.append(right[j]); j += 1
        res.extend(left[i:]); res.extend(right[j:])
        return res
    while len(sorted_chunks) > 1:
        next_chunks = []
        for i in range(0, len(sorted_chunks), 2):
            if i + 1 < len(sorted_chunks):
                next_chunks.append(merge_two(sorted_chunks[i], sorted_chunks[i + 1]))
            else:
                next_chunks.append(sorted_chunks[i])
        sorted_chunks = next_chunks
    return sorted_chunks[0]
if __name__ == '__main__':
```

```
N = 500_000
print(f"=== BENCHMARK QUICKSORT PARALELO (N = {N:,}) ===")
dados = [random.randint(1, 5_000_000) for _ in range(N)]
t_start = time.perf_counter()
res_seq = quicksort_seq(dados.copy())
t_seq = time.perf_counter() - t_start
print(f"[Sequencial] Tempo: {t_seq:.4f}s")
t_start = time.perf_counter()
res_par = parallel_quicksort(dados, num_workers=4)
t_par = time.perf_counter() - t_start
speedup = t_seq / t_par if t_par > 0 else 0
print(f"[Paralelo 4 Cores] Tempo: {t_par:.4f}s | Speedup: {speedup:.2f}x")
assert res_seq == res_par, "Erro na ordenação!"
print("✓ Validação: Ordenação paralela validada com sucesso!")
```

3. oddeven_sort_paralelo.py

Odd-Even Transposition Sort (BubbleSort Paralelo)

Objetivo: Mostrar como adaptar o algoritmo de bolha (BubbleSort), tradicionalmente sequencial, para um modelo paralelo.

Funcionamento: A ordenação ocorre em duas fases alternadas repetidas por N iterações: a Fase Par (Even), que compara e troca pares nos índices (0,1), (2,3), (4,5), e a Fase Ímpar (Odd), que compara pares nos índices (1,2), (3,4), (5,6).

Conceito-Chave: Como as comparações em cada fase são totalmente independentes entre si, todos os pares de uma mesma fase podem ser trocados em paralelo sem risco de Condição de Corrida (Race Condition).

```
# 03_oddeven_sort_paralelo.py

# Computação Paralela e Concorrente — Aulas 09

# Odd-Even Transposition Sort (Bubble Sort Paralelo em Fases Par/Ímpar)

import time

import random

def odd_even_sort_seq(arr):

    n = len(arr)

    is_sorted = False

    while not is_sorted:

        is_sorted = True

        # Fase Ímpar (Odd)

        for i in range(1, n - 1, 2):

            if arr[i] > arr[i + 1]:

                arr[i], arr[i + 1] = arr[i + 1], arr[i]

                is_sorted = False

        # Fase Par (Even)

        for i in range(0, n - 1, 2):
```

```
            if arr[i] > arr[i + 1]:
                arr[i], arr[i + 1] = arr[i + 1], arr[i]
                is_sorted = False
    return arr
if __name__ == '__main__':
    N = 5_000
    print(f"=== ODD-EVEN TRANSPOSITION SORT (N = {N:,}) ===")
    dados = [random.randint(1, 100_000) for _ in range(N)]
    t_start = time.perf_counter()
    res = odd_even_sort_seq(dados.copy())
    t_total = time.perf_counter() - t_start
    print(f"Tempo de Execução (Odd-Even): {t_total:.4f}s")
    assert res == sorted(dados), "Erro na ordenação!"
    print("✓ Validação: Array ordenado corretamente!")
```

4. bitonic_sort_paralelo.py

Bitonic Sort (Redes de Ordenação)

Objetivo: Apresentar o conceito de **Redes de Ordenação (Sorting Networks)** e padrão de comparação estático.

Funcionamento: O algoritmo converte a sequência de dados em uma **Sequência Bitônica** (primeira metade crescente, segunda decrescente) e aplica sucessivas etapas de comparação e troca cruzada em um padrão conhecido como mariposa (butterfly).

Conceito-Chave: Por ter um fluxo de execução predeterminado e independente dos valores dos dados, é a estrutura ideal para **Paralelismo de Dados (SIMD)** em hardware massivamente paralelo, como **GPUs (CUDA)** e arquiteturas vetoriais.

```
# 04_bitonic_sort_paralelo.py

# Computação Paralela e Concorrente — Aulas 09

# Bitonic Sort (Rede de Ordenação Paralela para N = 2^k)

import time

import random

def comp_and_swap(arr, i, j, direc):

    if (direc == 1 and arr[i] > arr[j]) or (direc == 0 and arr[i] < arr[j]):

        arr[i], arr[j] = arr[j], arr[i]

def bitonic_merge(arr, low, cnt, direc):

    if cnt > 1:

        k = cnt // 2

        for i in range(low, low + k):

            comp_and_swap(arr, i, i + k, direc)

        bitonic_merge(arr, low, k, direc)

        bitonic_merge(arr, low + k, k, direc)

def bitonic_sort_rec(arr, low, cnt, direc):

    if cnt > 1:

        k = cnt // 2
```

```
        bitonic_sort_rec(arr, low, k, 1) # Ordem crescente
        bitonic_sort_rec(arr, low + k, k, 0) # Ordem decrescente
        bitonic_merge(arr, low, cnt, direc)
def bitonic_sort(arr):
    bitonic_sort_rec(arr, 0, len(arr), 1)
    return arr
if __name__ == '__main__':
    N = 8192  # 2^13 elementos
    print(f"=== BITONIC SORT (REDE DE ORDENAÇÃO N = {N:,}) ===")
    dados = [random.randint(1, 100_000) for _ in range(N)]
    t_start = time.perf_counter()
    res = bitonic_sort(dados.copy())
    t_total = time.perf_counter() - t_start
    print(f"Tempo de Execução (Bitonic Sort): {t_total:.4f}s")
    assert res == sorted(dados), "Erro na ordenação!"
    print("✓ Validação: Sequência bitônica mesclada e ordenada!")
```

5. benchmark_ordenacao.py

Suíte Automática de Benchmarking e Desempenho

Objetivo: Coletar métricas reais de desempenho e validar a corretude dos algoritmos paralelos na prática.

Funcionamento: Gera vetores de dados aleatórios de tamanhos crescentes ($N = 10.000, 100.000, 500.000, 1.000.000$), executa o algoritmo sequencial e o paralelo sobre o mesmo conjunto de dados e compara as saídas.

Conceito-Chave: Valida a corretude via assert e calcula automaticamente o **Speedup (**S = T_{serial}/{T_{paralelo}**)** e a **Eficiência (**E% = {S}{p} x 100**)**, permitindo ao aluno visualizar na prática o ponto em que o tempo computacional supera o *overhead* de paralelização.

Como ele funciona?

Ele é um script **autônomo** (*standalone*). Ao ser executado no VS Code, ele:

1. Gera automaticamente vetores com números aleatórios em 4 tamanhos de carga crescentes (N = 10.000, 100.000, 500.000 e 1.000.000 de elementos).
2. Executa a versão sequencial e a versão paralela sobre a mesma massa de dados.
3. Calcula e exibe automaticamente no terminal o tempo de relógio, o **Speedup (**S**)** e a **Eficiência (**E\%**)** para cada tamanho de entrada.

Qual é o momento ideal de usá-lo na aula prática?

A ordem pedagógica recomendada no roteiro de laboratório é:

1. **Início da Prática:** Os alunos rodam e analisam os scripts individuais (*mergesort_paralelo.py*, *quicksort_paralelo.py*, etc.) para entenderem como cada algoritmo faz o particionamento, as trocas e a sincronização.
2. **Final/Coleta de Dados:** Os alunos executam o ***benchmark_ordenacao.py*** para obter a tabela completa de medições e preencher a **Folha de Respostas/Relatório de Laboratório**.

```
# 05_benchmark_ordenacao.py

# Benchmark Completo dos Algoritmos de Ordenação Paralelos e Seriais

# Computação Paralela e Concorrente (UNIFOR)

import time
```

```
import random
import os
import math
from concurrent.futures import ProcessPoolExecutor
def merge(left, right):
    res, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            res.append(left[i]); i += 1
        else:
            res.append(right[j]); j += 1
    res.extend(left[i:]); res.extend(right[j:])
    return res
def merge_sort_seq(arr):
    if len(arr) <= 1: return arr
    m = len(arr) // 2
    return merge(merge_sort_seq(arr[:m]), merge_sort_seq(arr[m:]))
def parallel_merge_sort(arr, num_workers=4):
    if len(arr) <= 10000:
        return merge_sort_seq(arr)
     chunk_size = math.ceil(len(arr) / num_workers)
    chunks = [arr[i:i + chunk_size] for i in range(0, len(arr), chunk_size)]

    with ProcessPoolExecutor(max_workers=num_workers) as executor:
        sorted_chunks = list(executor.map(merge_sort_seq, chunks))
    while len(sorted_chunks) > 1:
        next_chunks = []
```

```
        for i in range(0, len(sorted_chunks), 2):
            if i + 1 < len(sorted_chunks):
                next_chunks.append(merge(sorted_chunks[i], sorted_chunks[i + 1]))
            else:
                next_chunks.append(sorted_chunks[i])
        sorted_chunks = next_chunks
    return sorted_chunks[0]
if __name__ == '__main__':
    cores = os.cpu_count() or 4
    tamanhos = [10000, 100000, 500000, 1000000]

    print("============================================================")
    print(f"  BENCHMARK DE ORDENACAO PARALELA - SISTEMA ({cores} CORES)")
    print("============================================================\n")
    for N in tamanhos:
        dados = [random.randint(1, 10000000) for _ in range(N)]
        # Serial
        t0 = time.perf_counter()
        res_seq = merge_sort_seq(dados.copy())
        t_seq = time.perf_counter() - t0
        # Paralelo
        t0 = time.perf_counter()
        res_par = parallel_merge_sort(dados, num_workers=cores)
        t_par = time.perf_counter() - t0
        speedup = t_seq / t_par if t_par > 0 else 0
        eficiencia = (speedup / cores) * 100
        print(f"Tamanho N = {N:,}:")
```

```
print(f"  - Tempo Serial   : {t_seq:.4f} s")
print(f"  - Tempo Paralelo : {t_par:.4f} s")
print(f"  - Speedup        : {speedup:.2f}x")
print(f"  - Eficiencia     : {eficiencia:.1f}%\n")
```